

Diseño de una secuencia didáctica asistida por inteligencia artificial

A continuación, se presenta una **secuencia didáctica universitaria estructurada**, diseñada para una sesión de **50 minutos** en la asignatura **Traductores** para estudiantes de **8.º semestre de la carrera de Ingeniería en Sistemas Computacionales**. La propuesta se fundamenta en literatura clásica de **teoría de compiladores y autómatas**, integrando **herramientas modernas de generación automática y uso responsable de IA**.

Secuencia didáctica: Traducción de AFN a AFD en el contexto de los traductores

1. Contexto de la clase

Asignatura: Traductores

Carrera: Ingeniería en Sistemas Computacionales

Semestre: Octavo

Duración: 50 minutos

Unidad temática: Análisis léxico y herramientas de generación automática

En la construcción de **traductores de lenguajes de programación** (compiladores e intérpretes), el **analizador léxico** es comúnmente generado a partir de **expresiones regulares**, que se convierten primero en **autómatas finitos no deterministas (AFN)** y posteriormente en **autómatas finitos deterministas (AFD)** para su implementación eficiente.

Herramientas ampliamente utilizadas como:

- Flex lexical analyzer generator
- GNU Bison parser generator
- ANTLR parser generator

utilizan estos fundamentos formales para **generar analizadores léxicos y sintácticos automáticamente**.

En este contexto, comprender la **traducción AFN → AFD (algoritmo de subconjuntos)** es fundamental para entender cómo funcionan estas herramientas y cómo se construyen los **componentes de un traductor moderno**.

2. Objetivos de aprendizaje (Taxonomía de Bloom)

Al finalizar la sesión, el estudiante será capaz de:

1. **Analizar** la estructura de un **AFN derivado de una expresión regular** e identificar sus posibles transiciones y cierres ϵ .
2. **Aplicar** el **algoritmo de construcción de subconjuntos** para **traducir un AFN a un AFD** y explicar su relación con el funcionamiento de generadores automáticos de analizadores léxicos.

Nivel cognitivo esperado: **Aplicación y Análisis** (Bloom revisada) [1].

3. Secuencia didáctica

3.1 Inicio (10 minutos)

Activación de conocimientos previos

El docente plantea la pregunta guía:

“Cuando escribimos reglas léxicas en Flex o ANTLR usando expresiones regulares, ¿cómo se ejecuta realmente esa detección de tokens dentro del programa generado?”

Se presenta el flujo conceptual del análisis léxico:

Expresión regular

↓

Construcción de AFN

↓

Traducción AFN → AFD

↓

Tabla de transición eficiente

↓

Analizador léxico (scanner)

Conceptos clave introducidos brevemente:

- **AFN (Autómata Finito No Determinista)**
- **AFD (Autómata Finito Determinista)**
- **Expresiones regulares**
- **Compilador vs intérprete**
- **Herramientas de generación automática**

Relación conceptual:

Concepto	Función en un traductor
Expresiones regulares	Definición de tokens
AFN	Representación intermedia
AFD	Implementación eficiente
Flex	Generador de analizadores léxicos
Bison / ANTLR	Generadores de analizadores sintácticos

3.2 Desarrollo (30 minutos)

Ejemplo práctico (no clásico de libro)

Expresión regular utilizada en un lenguaje de configuración:

on|off

Esta expresión reconoce **palabras clave booleanas** utilizadas en archivos de configuración.

AFN propuesto

Estados:

$q_0 \xrightarrow{\epsilon} q_1$

$q_0 \xrightarrow{\epsilon} q_4$

$q_1 \xrightarrow{o} q_2$

$q_2 \xrightarrow{n} q_3$ (aceptación)

$q_4 \xrightarrow{o} q_5$

$q_5 \xrightarrow{f} q_6$

$q_6 \xrightarrow{f} q_7$ (aceptación)

Paso 1 – Cierre epsilon

$\epsilon\text{-closure}(q_0)$:

$\{q_0, q_1, q_4\}$

Este conjunto será el **estado inicial del AFD**.

Paso 2 – Construcción de subconjuntos

Estado A = {q0,q1,q4}

Transiciones:

Símbolo Resultado

o {q2,q5}

Estado B = {q2,q5}

Símbolo Resultado

n {q3}

f {q6}

Estado C = {q3}

Estado de aceptación.

Estado D = {q6}

Símbolo Resultado

f {q7}

Estado E = {q7}

Estado de aceptación.

Tabla final del AFD

Estado o n f Aceptación

A	B – –
B	– C D
C	– – – ✓
D	– – E
E	– – – ✓

Relación con herramientas reales

Ejemplo de regla en Flex:

```
%%  
on { return TOKEN_ON; }  
off { return TOKEN_OFF; }  
%%
```

Internamente Flex:

1. Convierte las reglas a **expresiones regulares**
 2. Construye **AFN**
 3. Genera **AFD**
 4. Produce código C optimizado.
-

3.3 Actividad integradora (10 minutos)

Ejercicio propuesto

Expresión regular:

yes|no

Tarea en equipos de 2 estudiantes:

1. Construir el **AFN** correspondiente.
2. Aplicar el **algoritmo de subconjuntos** para obtener el **AFD**.
3. Generar la **tabla de transición**.

Cierre de la actividad

Dos equipos realizan una **exposición oral breve (2 minutos)** explicando:

- Estados obtenidos
- Decisiones tomadas en el cierre ε
- Dificultades encontradas

4. Recursos educativos con apoyo de IA

Recurso 1 — Generación de diagrama con Graphviz

Los estudiantes pueden generar el AFN con:

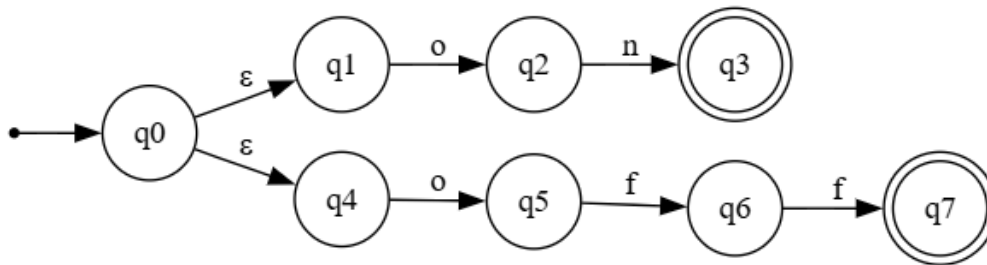
```
digraph AFN_on_off {  
  
    rankdir=LR;  
  
    node [shape=circle];  
  
    start [shape=point];  
  
    q0;  
    q1;  
    q2;  
    q3;  
    q4;  
    q5;  
    q6;  
    q7 [shape=doublecircle];  
  
    q3 [shape=doublecircle];  
  
    start -> q0;
```

```
q0 -> q1 [label="ε"];
q0 -> q4 [label="ε"];

q1 -> q2 [label="o"];
q2 -> q3 [label="n"];

q4 -> q5 [label="o"];
q5 -> q6 [label="f"];
q6 -> q7 [label="f"];

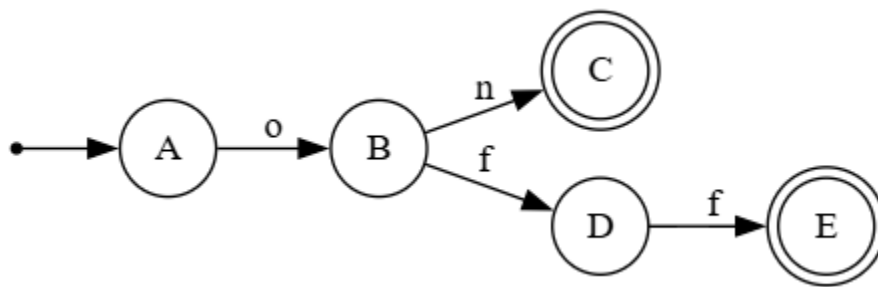
}
```



Y el AFD

```
digraph AFD {
rankdir=LR;
node [shape=circle];
A -> B [label="o"];
B -> C [label="n"];
B -> D [label="f"];
D -> E [label="f"];
```

```
inicio [shape=point];  
inicio -> A;  
C [shape=doublecircle];  
E [shape=doublecircle];  
}
```



Evaluación

Criterio	Evaluación
Confiabilidad	Alta, si se verifican manualmente las transiciones
Sesgos	Nulos (representación formal)
Derechos de autor	Libre (Graphviz es open-source)
Pertinencia	Alta, ya que los elementos tienen alta correspondencia al área

Recurso 2 — Verificación de cierres ϵ mediante IA

La IA puede ayudar a:

- Enumerar estados alcanzables
- Verificar subconjuntos generados

Riesgo:

La IA puede **omitir estados o aceptar transiciones incorrectas** si el AFN está mal descrito.

Recurso 3 — Generación automática de tabla de transición

Ejemplo de prompt:

Construye la tabla de transición del AFD obtenido del siguiente AFN...

Tabla de transición del AFD

```
digraph TablaTransicion {
node [shape=none fontname="Arial"];
tabla [label=<
<table border="1" cellborder="1" cellspacing="0">
<tr>
<td><b>Estado</b></td>
<td><b>o</b></td>
<td><b>n</b></td>
<td><b>f</b></td>
</tr>

<tr>
<td>> A</td>
<td>B</td>
<td>∅</td>
<td>∅</td>
</tr>
}
```

<tr>

<td>B</td>

<td>Ø</td>

<td>C</td>

<td>D</td>

</tr>

<tr>

<td>*C</td>

<td>Ø</td>

<td>Ø</td>

<td>Ø</td>

</tr>

<tr>

<td>D</td>

<td>Ø</td>

<td>Ø</td>

<td>E</td>

</tr>

<tr>

<td>*E</td>

<td>Ø</td>

```
<td>∅</td>

<td>∅</td>

</tr>

</table>

>];

}
```

Estado	o	n	f
→ A	B	∅	∅
B	∅	C	D
*C	∅	∅	∅
D	∅	∅	E
*E	∅	∅	∅

Evaluación:

Criterio **Evaluación**

Confiabilidad Media

Sesgo Posibles errores formales

Uso pedagógico Adecuado para verificación

5. Evaluación del aprendizaje

Evidencias

1. **AFN construido correctamente**
2. **Tabla de transición del AFD**
3. **Explicación oral del procedimiento**

Rúbrica simplificada

Criterio	Puntos
Construcción del AFN	3
Aplicación del algoritmo de subconjuntos	4
Tabla del AFD	2
Explicación oral	1

Total: **10 puntos**

6. Consideraciones éticas en el uso de IA

El uso de IA en esta actividad debe cumplir principios de **integridad académica**:

- La IA puede **apoyar la verificación**, no sustituir el razonamiento.
- Los estudiantes deben **documentar el procedimiento seguido**.
- Los resultados generados por IA deben **contrastarse con teoría formal**.

De acuerdo con lineamientos recientes de educación superior sobre IA [2], el aprendizaje profundo ocurre cuando el estudiante **explica el proceso**, no solo presenta el resultado.

7. Reflexión pedagógica final

Esta actividad de desarrolló de una secuencia didáctica se enfocó en los niveles de **Analizar y aplicar**. Se **analizó** el proceso de traducción de un AFN a un AFD **aplicando** el algoritmo de construcción de subconjuntos.

Fortalezas

Taller: Cómo diseñar clases efectivas con IA educativa

- Se utilizó la IA para apoyar la generación de esta secuencia, por lo que el tiempo para formar obtener este resultado fue extremadamente mas rápido que haberla realizado sin ella. Esto permitió utilizar otras herramientas que me permitieron enriquecer la secuencia. El tema elegido es un tema clásico en el área de Traductores por lo que me fue mas familiar identificar la calidad del contenido propuesto y los elementos que podría agregar a la secuencia. Por lo general prefiero utilizar un prompt base y trabajar manualmente para curar el contenido.
- Agilizar la creación de la secuencia y sus contenidos permite profundizar o alcanzar mejor los objetivos propuestos.
- El uso de la IA promueve el uso de otras herramientas de uso específico. En mi caso la IA no genera directamente la figura que hay que desplegar, pero te permite la generación del código a utilizar.

Posibles debilidades

- La facilidad con la que la IA provee los resultados en la solución de problemas puede promover un aprendizaje superficial en áreas que requieren una comprensión profunda como en áreas de algoritmos y sistemas de traducción. La actividad práctica podría necesitar refuerzo posterior.

Decisiones que dependen del docente

Específicamente en la secuencia presentada, el docente debe decidir:

- Nivel de complejidad del AFN.
- Profundidad matemática del algoritmo.
- Grado de uso permitido de IA.

Actividades que puede apoyar la IA

- Generación de diagramas
- Verificación de tablas
- Visualización de autómatas

Actividades que deben realizar los estudiantes

- Construcción del AFN, lo cual requiere del uso de varios niveles de la escala de Bloom.
- Aplicación del algoritmo de subconjuntos paso a paso.
- Interpretación de resultados para verificar que el resultado es correcto.

Estas tareas desarrollan **pensamiento computacional y comprensión formal**, competencias clave en la formación de ingenieros.

Reflexión personal

La ayuda que las herramientas de IA proporcionan es muy grande, de tal manera que es muy fácil abusar de sus beneficios y no desarrollar las habilidades y competencias que se buscan. Pero a pesar de esto quiero decir que el desarrollo de esta secuencia me permitió visualizar la clase desde la perspectiva del docente y del ingeniero en sistemas. La IA puede generar mucho contenido en un abrir y cerrar de ojos, pero se requiere poner atención para validar y revisar los contenidos generados. Agradezco al instructor del Taller “Cómo diseñar clases efectivas con IA educativa” por compartirnos ideas y herramientas tan importantes para mejorar nuestro trabajo docente.

Referencias (formato IEEE)

- [1] L. W. Anderson y D. R. Krathwohl, *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. New York: Longman, 2001.
- [2] UNESCO, *Guidance for Generative AI in Education and Research*. Paris: UNESCO, 2023.
- [3] A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston: Pearson, 2006.
- [4] J. E. Hopcroft, R. Motwani y J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Boston: Pearson, 2006.
- [5] J. R. Levine, T. Mason y D. Brown, *Lex & Yacc*, 2nd ed. Sebastopol: O'Reilly Media, 1992.
- [6] T. Parr, *The Definitive ANTLR 4 Reference*. Dallas: Pragmatic Bookshelf, 2013.

APENDICES

A. Prompt base utilizado.

Actúa como experto en diseño instruccional universitario y en el área de traductores e intérpretes

Diseña una secuencia didáctica completa para una clase de educación superior de 50 minutos en la materia de Traductores, cuyo tema sea traducción de un AFN a un AFD. La clase está dirigida a alumnos de la Ingeniería en Sistemas Computacionales del octavo semestre. En la secuencia incluye los conceptos de AFN, AFD, expresiones regulares, flex, antlr, bison, traductores, compiladores, intérpretes y herramientas de generación automática. Concluya la secuencia con exposición oral breve de la solución de un ejercicio de traducción propuesto y los detalles enfrentados.

El material generado debe contar con las siguientes características:

Rigor conceptual (académicamente fundamentado)

Evidencias y fuentes verificables

Sin sesgos o generalizaciones sin respaldo

Pertinencia pedagógica hacia objetivos formativos claros

Ética académica del entorno universitario

Incluya citas, referencias y la bibliografía en formato IEEE de los contenidos generados

Desarrolla la propuesta siguiendo las siguientes etapas pedagógicas:

Etapas 1 – Generación de la secuencia inicial

Define el contexto de la clase.

Propón una secuencia didáctica inicial que incluya inicio, desarrollo y actividad integradora.

Integra ejemplos prácticos de aplicación de la traducción de AFN a AFD pero que no sean los clásicos del libro de texto de Holub o Aho.

Etapas 2 – Coherencia pedagógica

Formula uno o dos objetivos de aprendizaje utilizando la taxonomía de Bloom.

Verifica que las actividades sean coherentes con el nivel cognitivo esperado.

Sugiere posibles ajustes para mejorar la claridad pedagógica.

Etapa 3 – Integración de recursos con IA

Propón dos o tres recursos educativos generados con apoyo de IA (por ejemplo: pasos de la traducción de un AFN a un AFD, verificación de los resultados en cada etapa, obtención de la tabla de transición o construcción del AFD con Graphviz).

Evalúa cada recurso considerando confiabilidad, posibles sesgos, derechos de autor y pertinencia educativa.

Etapa 4 – Reflexión docente

Identifica fortalezas de la secuencia didáctica.

Señala posibles debilidades o limitaciones.

Explica qué decisiones pedagógicas deben depender exclusivamente del criterio del docente, que secciones pueden ser resueltas usando IA y las que deben realizarse necesariamente por el alumno.

Presenta el resultado en formato estructurado con los siguientes apartados:

Contexto de la clase

Objetivos de aprendizaje

Secuencia didáctica

Recursos educativos

Evaluación del aprendizaje

Consideraciones éticas en el uso de IA

Reflexión pedagógica final

B. Prompt para curación

Revisión crítica:

Verifica que AFN y AFD propuestos contengan los estados de inicio y aceptación y generen también la tabla de transiciones resultante.

Reescritura fundamentada:

Reescribe el autómata en formato .dot agregando estados iniciales y finales de los autómatas propuestos.

Adaptación contextual:

Utiliza la nomenclatura utilizada en el libro de Aho, Ullman para los estados de los autómatas y para la teoría de la construcción de subconjuntos